



Taller práctico de Git

Crear una simulación de “página web” para una cafetería llamada Café Aurora.

No usaremos código complejo.

El objetivo es aprender Git paso a paso como en un entorno profesional.

Usaremos:

- ramas
- commits
- merge
- stash
- tags
- remotos
- conflictos
- restauración
- historial
- rebase
- etc.

Caso práctico

Tiene esta carpeta: `cafe-aurora/`

Dentro habrá archivos simples:

```
index.html  
menu.txt  
contacto.txt  
README.md
```

PARTE 1 Crear repositorio

1) git init

Inicializa Git.

```
git init
```

Resultado:

```
Initialized empty Git repository
```

```
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~ ((4a2153c...))  
$ git init  
Reinitialized existing Git repository in C:/Users/Aprendiz/.git/
```

Ing. Víctor Rincón



2) git status

Ver estado actual.

git status

```
aprendiz@DESKTOP-LPU1C05 MINGW64 ~ ((4a2153c...))
$ git status
warning: could not open directory 'Configuración local/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Datos de programa/': Permission denied
warning: could not open directory 'Entorno de red/': Permission denied
warning: could not open directory 'Impresoras/': Permission denied
warning: could not open directory 'Menú Inicio/': Permission denied
warning: could not open directory 'Mis documentos/': Permission denied
warning: could not open directory 'Plantillas/': Permission denied
warning: could not open directory 'Reciente/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
HEAD detached at 4a2153c
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    .ld9virtualbox/
    .bash_history
    .git-for-windows-updater
    .gitconfig
    .idlerc/
    .lessht
    .nbi/
    .node_repl_history
    .openjfx/
    .vscode-shared/
    .vscode/
    AppData/
    Contacts/
    Desktop/
    Documents/
    Downloads/
    Favorites/
    IntelGraphicsProfiles/
    Links/
    Music/
    NTUSER.DAT
    NTUSER.DAT{d350bf1c-fd4a-11ef-9e70-e86f38a40c6e}.TM.blf
    NTUSER.DAT{d350bf1c-fd4a-11ef-9e70-e86f38a40c6e}.TMContainer00000000000000000001.regtrans-ms
    NTUSER.DAT{d350bf1c-fd4a-11ef-9e70-e86f38a40c6e}.TMContainer00000000000000000002.regtrans-ms
    OneDrive/
    Pictures/
    Saved Games/
    Searches/
    Videos/
    jamer/
    node_modules/
    ntuser.dat.LOG1
    ntuser.dat.LOG2
    ntuser.ini
    package-lock.json
    package.json
    pseint/

nothing added to commit but untracked files present (use "git add" to track)
```

3) git config



Configurar usuario.

```
git config --global user.name "Juan"  
git config --global user.email "juan@email.com"
```

Ver configuración:

```
git config --list
```

```
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~ ((4a2153c...))  
$ git config --global user.name "Jeann"  
  
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~ ((4a2153c...))  
$ git config --global user.mail "jeannmarcorodriguez@gmail.com"  
  
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~ ((4a2153c...))  
$ git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=schannel  
core.autocrlf=input  
core.fsckache=true  
core.smlinks=false  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
user.name=Jeann  
user.email=jeannmarcorodriguez@gmail.com  
user.mail=jeannmarcorodriguez@gmail.com  
core.repositoryformatversion=0  
core.filemode=false  
core.bare=false  
core.logallrefupdates=true  
core.smlinks=false  
core.ignorecase=true
```

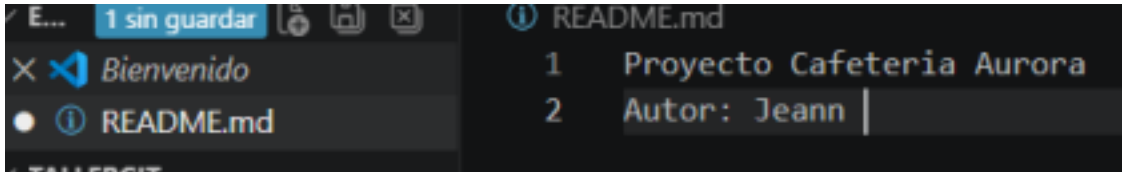
PARTE 2 Primer commit

Crea:

README.md

Contenido:

Proyecto cafetería Cafe Aurora



4) git add

```
git add README.md
```

```
AprenDiz@DESKTOP-LPU1C05 MINGW64 ~/tallergit ((4a2153c...))
$ git add README.md
```

git status

5) git status

```
git status
```

Verás:

Changes to be committed

```
$ git status
warning: could not open directory 'Configuración local/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Datos de programa/': Permission denied
warning: could not open directory 'Entorno de red/': Permission denied
warning: could not open directory 'Impresoras/': Permission denied
warning: could not open directory 'Menú Inicio/': Permission denied
warning: could not open directory 'Mis documentos/': Permission denied
warning: could not open directory 'Plantillas/': Permission denied
warning: could not open directory 'Reciente/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
HEAD detached at 4a2153c
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   README.md

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   README.md
```

6) git commit

```
git commit -m "Primer commit"
```

```
git
```

```
$ git commit -m "Primer commit"
[detached HEAD 1a8c7be] Primer commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 tallergit/README.md
```

git st

7) git log

```
git
```

```
git log
```



Ing. Víctor Rincón

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit ((a31340e...))
$ git log
commit a31340e72f291a955d47c63dcae1f21faa0b2845 (HEAD)
Author: Jeann <jeannmarcorodriguez@gmail.com>
Date: Tue May 26 21:16:44 2026 -0500

    Primer commit

commit 1a8c7be0fb1d21359b7ebbf44961b6d2b673ed21
Author: Jeann <jeannmarcorodriguez@gmail.com>
Date: Tue May 26 21:11:28 2026 -0500

    Primer commit

commit 4a2153ca1f191aadfcb7fcab89d6d01f09430aa5 (master)
Author: Makro <jeannmarcorodriguez@gmail.com>
Date: Mon May 25 20:57:29 2026 -0500

    primer commit del proyecto
```

8) git log --oneline

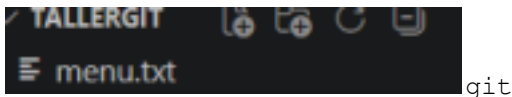
git log --oneline

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git log --oneline
ecede17 (HEAD -> master) Primer commit
4a2153c primer commit del proyecto
```

git PARTE 3 Trabajar con archivos

Crea:

menu.txt



git

9) git add .

git add .

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git add menu.txt
```

10) git diff

Modifica menu.txt.

Ing. Víctor Rincón



Luego:

```
git diff
```

Muestra cambios no staged.git git commit

```
$ git diff
diff --git a/tallergit/menu.txt b/tallergit/menu.txt
index e69de29..5ba0ce9 100644
--- a/tallergit/menu.txt
+++ b/tallergit/menu.txt
@@ -0,0 +1 @@
+Menu Cafeteria
\ No newline at end of file
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git diff
diff --git a/tallergit/menu.txt b/tallergit/menu.txt
index 307d515..3720b59 100644
--- a/tallergit/menu.txt
+++ b/tallergit/menu.txt
@@ -1,1 @@
-Menu Cafeteria menu
\ No newline at end of file
+Menu Cafeteria m
\ No newline at end of file
```

11) git commit

```
git commit -m "Agrega menú inicial"
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git commit -m "Agrega menu inicial"
[master df430ea] Agrega menu inicial
1 file changed, 1 insertion(+)
create mode 100644 tallergit/menu.txt
```

PARTE 4 Branches

12) git branch

```
git branch
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit ((f4af34b...))
$ gir branch
bash: gir: command not found

Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit ((f4af34b...))
$ git branch
* (HEAD detached from 4a2153c)
master
```

```
gir
```

13) git branch feature/contacto

```
git branch feature/contacto
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git branch feature/contacto
```

14) git checkout



```
aprendiz@DESKTOP-LPUICOS MINGW64 ~/tallergit ((f4af34b...))
$ git checkout feature
#       tallergit/menu.txt
Switched to branch 'feature'
```

15) git switch

git s

```
aprendiz@DESKTOP-LP0ICUS MINGW64 ~/tailergit (feature)
$ git stash
Saved working directory and index state WIP on feature: f4af34b Agrega menu inicial
```

Volver a main:

```
git checkout master git
```

```
Aprendiz@DESKTOP-LP0IC05 MINGW64 ~/tallergit (feature)
$ git checkout master
Switched to branch 'master'
```

16) git checkout -b

Crear y cambiar:

```
git checkout -b feature/redes-sociales
```

```
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~/tallergit (master)
$ git checkout -b feature/redes-sociales
Switched to a new branch 'feature/redes-sociales'
```

PARTE 5 Merge

Agrega:

instagram: @cafeaurora

Commit:

```
git add .
git commit -m "Agrega redes sociales" git
```

17) git merge

Volver a main:

```
git switch main
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~  
$ git switch master  
Switched to branch 'master'
```

Fusionar:

Ing. Víctor Rincón



```
git merge feature/redes-sociales
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git merge feature/redes-sociales  
Updating df430ea..de436f0  
Fast-forward  
tallergit/README.md | 4 +++-  
1 file changed, 3 insertions(+), 1 deletion(-)
```

PARTE 6 Eliminar y mover archivos

18) git rm

```
git rm contacto.txt
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git rm contacto.txt  
rm 'tallergit/contacto.txt'
```

19) git mv

```
git mv menu.txt menu-cafeteria.txt
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/talle  
$ git mv menu.txt menu-cafeteria.txt
```

20) git commit

```
git commit -m "Reorganiza archivos"
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git commit -m "Reorganiza archivos"  
[master 2048fa8] Reorganiza archivos  
2 files changed, 0 insertions(+), 0 deletions(-)  
delete mode 100644 tallergit/contacto.txt  
rename tallergit/{menu.txt => menu-cafeteria.txt} (100%)
```


PARTE 7 Restaurar cambios

Modifica README.

21) git restore

```
git restore README.md
```

Descarta cambios.

```
Aprendiz@DESKTOP-LPU1C0S MINGW  
$ git restore README.md
```

22) git restore --staged

```
git restore --staged README.md
```

Quita del staging.

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/  
$ git restore --staged README.md
```

PARTE 8 Trabajar con remoto (GitHub)

23) git remote add

```
git remote add origin https://github.com/juan/cafe-aurora.git
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git remote add origin https://github.com/juan/cafe-aurora.git
```

24) git remote -v

```
git remote -v
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git remote -v  
origin https://github.com/juan/cafe-aurora.git (fetch)  
origin https://github.com/juan/cafe-aurora.git (push)
```

Ing. Víctor Rincón



25) git push

`git push -u origin main`

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git push -u origin master
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

26) git pull

`git pull`

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git pull
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

27) git fetch

`git fetch`

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git fetch
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

28) git clone

Otro desarrollador haría:

`git clone https://github.com/juan/cafe-aurora.git`

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git clone https://github.com/juan/cafe-aurora.git
Cloning into 'cafe-aurora'...
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

PARTE 9 Stash

Estás trabajando y necesitas cambiar de rama rápido.

29) git stash

`git stash`

Guarda temporalmente cambios.

```
Aprendiz@DESKTOP-LPU1C0S MINGW64
$ git stash
No local changes to save
```

30) git stash list

git stash list

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git stash list
stash@{0}: WIP on feature: f4af34b Agrega menu inicial
```

31) git stash pop

git stash pop

Recupera cambios.

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git stash pop
CONFLICT (modify/delete): tallergit/menu.txt deleted in Updated upstream and
modified in Stashed changes. Version Stashed changes of tallergit/menu.txt left
tree.
warning: could not open directory 'Configuración local/': Permission denied
warning: could not open directory 'Cookies/': Permission denied
warning: could not open directory 'Datos de programa/': Permission denied
warning: could not open directory 'Entorno de red/': Permission denied
warning: could not open directory 'Impresoras/': Permission denied
warning: could not open directory 'Menú Inicio/': Permission denied
warning: could not open directory 'Mis documentos/': Permission denied
warning: could not open directory 'Plantillas/': Permission denied
warning: could not open directory 'Reciente/': Permission denied
warning: could not open directory 'SendTo/': Permission denied
On branch master
Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add/rm <file>..." as appropriate to mark resolution)
    deleted by us:   menu.txt
```

PARTE 10 Historial y detalles

32) git show

git show

Ing. Víctor Rincón



```

Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git show
commit 2048fa89b1020921f20192ad32bc346bc6d10cc1 (HEAD -> master)
Author: Jeann <jeannmarcorodriguez@gmail.com>
Date: Thu May 28 20:18:18 2026 -0500

    Reorganiza archivos

diff --git a/tallergit/contacto.txt b/tallergit/contacto.txt
deleted file mode 100644
index e69de29..0000000
diff --git a/tallergit/menu.txt b/tallergit/menu-cafeteria.txt
similarity index 100%
rename from tallergit/menu.txt
rename to tallergit/menu-cafeteria.txt

```

33) git reflog

git reflog

Historial interno.

```

Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git reflog
2048fa8 (HEAD -> master) HEAD@{0}: commit: Reorganiza archivos
d347cc5 HEAD@{1}: commit: Creación de archivos iniciales para la práctica
de436f0 (feature/redes-sociales) HEAD@{2}: merge feature/redes-sociales: Fast-forward
df430ea (feature/contacto) HEAD@{3}: checkout: moving from feature/redes-sociales to master
de436f0 (feature/redes-sociales) HEAD@{4}: commit: Agrega redes sociales
df430ea (feature/contacto) HEAD@{5}: checkout: moving from master to feature/redes-sociales
df430ea (feature/contacto) HEAD@{6}: checkout: moving from feature/contacto to master
df430ea (feature/contacto) HEAD@{7}: checkout: moving from master to feature/contacto
df430ea (feature/contacto) HEAD@{8}: checkout: moving from feature to master
f4af34b HEAD@{9}: checkout: moving from feature to feature
f4af34b HEAD@{10}: checkout: moving from master to feature
df430ea (feature/contacto) HEAD@{11}: commit: Agrega menu inicial
ecede17 HEAD@{12}: commit: Primer commit

```

34) git blame

git blame README.md

Quién modificó cada línea.

```

Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)
$ git blame README.md
ecede17e (Jeann 2026-05-28 19:24:45 -0500 1) Proyecto Cafeteria Aurora
de436f03 (Jeann 2026-05-28 20:00:48 -0500 2) Autor: Jeann
de436f03 (Jeann 2026-05-28 20:00:48 -0500 3)
de436f03 (Jeann 2026-05-28 20:00:48 -0500 4) instagram: @cafeaurora

```

PARTE 11 Tags y versiones

35) git tag

```
git tag v1.0
```

```
Aprendiz@DESKTOP-LPU1  
$ git tag v1.0
```

36) git tag -a

```
git tag -a v1.1 -m "Versión estable"
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallerg  
$ git tag -a v1.1 -m "Versión estable"
```

37) git push --tags

```
git push --tags
```

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git push --tags  
info: please complete authentication in your browser...  
remote: Repository not found.  
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

PARTE 12 Reset y revert

38) git reset

```
git reset HEAD README.md
```

Quitar del staging.

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git reset HEAD README.md  
Unstaged changes after reset:  
U      tallergit/menu.txt
```

39) git revert

```
git revert HEAD
```

Deshacer commit sin borrar historial.

```
Aprendiz@DESKTOP-LPU1C0S MINGW64 ~/tallergit (master)  
$ git revert HEAD  
error: Reverting is not possible because you have unmerged files.  
hint: Fix them up in the work tree, and then use 'git add/rm <file>'  
hint: as appropriate to mark resolution and make a commit.  
fatal: revert failed
```



PARTE 13 Rebase

40) git rebase

```
git rebase main
```

Reorganiza historial.

```
Aprendiz@DESKTOP-LPU1C05 MINGW64 ~/taller/git (master)
$ git rebase master
Current branch master is up to date.
```

EXTRA Flujo profesional

```
git pull
git switch -c feature/menu
git add .
git commit -m "Actualiza menú"
git push origin feature/menu
```

RESUMEN VISUAL

```
Working Directory
  ↓
git add
  ↓
Staging Area
  ↓
git commit
  ↓
Repositorio local
  ↓
git push
  ↓
GitHub
```

LOS 40 COMANDOS APRENDIDOS

- 1 git init
- 2 git status

3 git config
4 git add
5 git commit
6 git log
7 git log --oneline
8 git diff
9 git branch
10 git checkout

Ing. Víctor Rincón



11 git switch
12 git merge
13 git rm
14 git mv
15 git restore
16 git remote
17 git push
18 git pull
19 git fetch
20 git clone
21 git stash
22 git stash list
23 git stash pop
24 git show
25 git reflog
26 git blame
27 git tag
28 git tag -a
29 git push --tags
30 git reset
31 git revert
32 git rebase
33 git checkout -b
34 git remote add
35 git restore --staged
36 git branch feature/*
37 git switch main
38 git merge feature/*
39 git add .
40 git commit -m

¿Te atreves a realizar el siguiente reto?

Una **simulación real de empresa**, donde usaras:

- comandos básicos (add, commit, branch...)
- comandos intermedios (stash, rebase, reset...)
- comandos avanzados (reflog, bisect, cherry-pick...)
- inspección interna de Git
- herramientas de debugging
- configuración
- limpieza

Ing. Víctor Rincón



- flujos de equipo

E-commerce Café Aurora pro

Trabaja en equipo con GitHub. Para desarrollar:

- tienda online
- pagos
- menú
- sistema de usuarios
- corrección de bugs



PARTE 1 CONFIGURACIÓN INICIAL (10 comandos)

```
git -version
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)  
$ git --version  
git version 2.54.0.windows.1
```

```
git config --global user.name "Juan" git config  
--global user.email "juan@email.com"
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)  
$ git config --global user.name "jonathan joya"
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)  
$ git config --global user.email "jonathanjoya642@gmail.com"
```

```
git config --list
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)  
$ git config --list  
diff.astextplain.textconv=astextplain  
filter.lfs.clean=git-lfs clean -- %f  
filter.lfs.smudge=git-lfs smudge -- %f  
filter.lfs.process=git-lfs filter-process  
filter.lfs.required=true  
http.sslbackend=schannel  
core.autocrlf=true  
core.fscache=true  
core.symlinks=false  
pull.rebase=false  
credential.helper=manager  
credential.https://dev.azure.com.usehttppath=true  
init.defaultbranch=master  
user.name=jonathan joya  
user.email=jonathanjoya642@gmail.com  
core.repositoryformatversion=0  
core.filemode=false  
core.bare=false  
core.logallrefupdates=true  
core.symlinks=false  
core.ignorecase=true  
remote.origin.url=https://github.com/juan/cafe-aurora.git  
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
```

```
git help
```



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]
```

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)

clone	Clone a repository into a new directory
init	Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)

add	Add file contents to the index
mv	Move or rename a file, a directory, or a symlink
restore	Restore working tree files
rm	Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)

bisect	Use binary search to find the commit that introduced a bug
diff	Show changes between commits, commit and working tree, etc
grep	Print lines matching a pattern
log	Show commit logs
show	Show various types of objects
status	Show the working tree status

grow, mark and tweak your common history

backfill	Download missing objects in a partial clone
branch	List, create, or delete branches
commit	Record changes to the repository
history	EXPERIMENTAL: Rewrite history
merge	Join two or more development histories together
rebase	Reapply commits on top of another base tip
reset	Set 'HEAD' or the index to a known state
switch	Switch branches
tag	Create, list, delete or verify tags

collaborate (see also: git help workflows)

fetch	Download objects and refs from another repository
pull	Fetch from and integrate with another repository or a local branch
push	Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some concept guides. See 'git help <command>' or 'git help <concept>' to read about a specific subcommand or concept. See 'git help git' for an overview of the system.

git help commit

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git help commit
```

git init

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git init
Reinitialized existing Git repository in C:/Users/jonat/TallerGIT/.git/
```

git clone <https://github.com/cafe/aurora.git>



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git clone https://github.com/cafe/aurora.git
Cloning into 'aurora'...
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/cafe/aurora.git/' not found
```

git remote -v git remote add origin URL

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git remote -v git remote add origin URL
error: unknown subcommand: 'git'
usage: git remote [-v | --verbose]
or: git remote add [-t <branch>] [-m <master>] [-f] [--tags | --no-tags] [--mirror=<fetch|push>] <name> <url>
or: git remote rename [--[no-]progress] <old> <new>
or: git remote remove <name>
or: git remote set-head <name> (-a | --auto | -d | --delete | <branch>)
or: git remote [-v | --verbose] show [-n] <name>
or: git remote prune [-n | --dry-run] <name>
or: git remote [-v | --verbose] update [-p | --prune] [(<group> | <remote>)...]
or: git remote set-branches [--add] <name> <branch>...
or: git remote get-url [--push] [--all] <name>
or: git remote set-url [--push] <name> <newurl> [<oldurl>]
or: git remote set-url --add <name> <newurl>
or: git remote set-url --delete <name> <url>

-v, --[no-]verbose    be verbose; must be placed before a subcommand
```

PARTE 2 INSPECCIÓN DEL PROYECTO (10 comandos)

git status

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git status
On branch main
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md
        contacto.txt
        menu-cafeteria.txt

nothing added to commit but untracked files present (use "git add" to track)
```

git status -s

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git status -s
?? README.md
?? contacto.txt
?? menu-cafeteria.txt
```

git log



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git log
commit 181bbe9e94803694fa4fafc7c47e6714a1a50852 (HEAD -> main)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date:   Sun May 31 13:40:37 2026 -0500

    Revert "primer commit"

    This reverts commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81.

commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81 (tag: v1.1, tag: v1.0)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date:   Sun May 31 13:32:20 2026 -0500

    primer commit
```

`git log --oneline`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git long --oneline
git: 'long' is not a git command. See 'git --help'.

The most similar commands are
    clone
    log
```

`git log --graph`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git log --graph
* commit 181bbe9e94803694fa4fafc7c47e6714a1a50852 (HEAD -> main)
  Author: jonathan joya <jonathanjoya642@gmail.com>
  Date:   Sun May 31 13:40:37 2026 -0500

      Revert "primer commit"

      This reverts commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81.

* commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81 (tag: v1.1, tag: v1.0)
  Author: jonathan joya <jonathanjoya642@gmail.com>
  Date:   Sun May 31 13:32:20 2026 -0500

      primer commit
```

`git log --all`



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git log --all
commit 181bbe9e94803694fa4fafc7c47e6714a1a50852 (HEAD -> main)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date: Sun May 31 13:40:37 2026 -0500

    Revert "primer commit"

    This reverts commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81.

commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81 (tag: v1.1, tag: v1.0)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date: Sun May 31 13:32:20 2026 -0500

    primer commit
```

git diff

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git diff
```

git diff HEAD

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git diff HEAD
```

git diff -staged

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git diff --staged
```

git show HEAD



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git show HEAD
commit 181bbe9e94803694fa4fafc7c47e6714a1a50852 (HEAD -> main)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date:   Sun May 31 13:40:37 2026 -0500

    Revert "primer commit"

    This reverts commit 41b7a35d1b600830ed2f4a3923fbbdcee991ed81.

diff --git a/README.md b/README.md
deleted file mode 100644
index 93dee76..0000000
--- a/README.md
+++ /dev/null
@@ -1,5 +0,0 @@
-autores:Anyelo, jeann,camilo,jonathan
-
-instagram: @cafeaurora
-
-antacto:1234567890
diff --git a/contacto.txt b/contacto.txt
deleted file mode 100644
index e69de29..0000000
diff --git a/menu-cafeteria.txt b/menu-cafeteria.txt
deleted file mode 100644
index 408c5a3..0000000
--- a/menu-cafeteria.txt
+++ /dev/null
@@ -1 +0,0 @@
-massss
\ No newline at end of file
```

PARTE 3 WORKFLOW BÁSICO (10 comandos)

git add archivo.txt

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git add archivos.txt
fatal: pathspec 'archivos.txt' did not match any files
```

git add .

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git add .
```

git add -A



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git add -A
```

`git commit -m "mensaje"`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git commit -m "mensaje"
[main b27123d] mensaje
3 files changed, 5 insertions(+)
create mode 100644 README.md
create mode 100644 contacto.txt
create mode 100644 menu-cafeteria.txt
```

`git commit -am "mensaje"`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git commit -am "mensaje"
On branch main
nothing to commit, working tree clean
```

`git rm archivo.txt`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ rm archivo.txt
```

`git mv viejo.txt nuevo.txt`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ mv viejo.txt nuevo.txt
```

`git restore archivo.txt`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git restore archivo.txt
```

`git restore --staged archivo.txt`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git restore --staged archivo.txt
```

`git clean -f`

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git clean -f
Removing archivo.txt
```



PARTE 4 RAMAS (12 comandos)

git branch

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git branch
  feature/contacto0
* main
```

git branch feature/login

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git branch feature/login
```

git branch -d feature/login

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git branch -d feature/login
Deleted branch feature/login (was 630d23d).
```

git branch -D feature/login

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git branch -D feature/login
Deleted branch feature/login (was 630d23d).
```

git checkout main

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git checkout main
A      nuevo.txt
Already on 'main'
```

git checkout -b feature/login

```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git checkout -b feature/login
Switched to a new branch 'feature/login'
```

git switch main

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/login)
$ git switch main
A      nuevo.txt
Switched to branch 'main'
```

git switch -c feature/login



```
jonat@F_Joya MINGW64 ~/tallerGIT (main)
$ git switch -c feature/login
Switched to a new branch 'feature/login'
```

`git merge feature/login`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/login)
$ git merge feature/login
Already up to date.
```

`git merge --no-ff feature/login`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/login)
$ git merge --no-ff feature/login
Already up to date.
```

`git branch -m nuevo-nombre`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/login)
$ git branch -m nuevo-nombre
```

`git branch -r`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git branch -r
```

PARTE 5 REBASE Y HISTORIAL (8 comandos)

`git rebase main`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git rebase main
Current branch nuevo-nombre is up to date.
```

`git rebase -i HEAD~3`

```
Successfully rebased and updated refs/heads/nuevo-nombre.

jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$
```



`git rebase --abort`

```
Successfully rebased and updated refs/heads/nuevo-nombre.  
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)  
$ git rebase --abort  
fatal: no rebase in progress  
  
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)  
$ |
```

`git rebase --continue`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)  
$ git rebase --continue  
fatal: no rebase in progress
```

`git cherry-pick abc1234`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)  
$ git cherry-pick 630d23d  
On branch nuevo-nombre  
You are currently cherry-picking commit 630d23d.  
  (all conflicts fixed: run "git cherry-pick --continue")  
  (use "git cherry-pick --skip" to skip this patch)  
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)  
  
nothing to commit, working tree clean  
The previous cherry-pick is now empty, possibly due to conflict resolution.  
If you wish to commit it anyway, use:  
  
    git commit --allow-empty  
  
Otherwise, please use 'git cherry-pick --skip'  
  
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre|CHERRY-PICKING)  
$ |
```

`git revert HEAD`



```
[nuevo-nombre 0176e3b] Revert "Mi primer commit con archivo.txt"
Date: Sun May 31 14:43:31 2026 -0500
1 file changed, 1 insertion(+)
create mode 100644 archivo.txt

jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$
```

`git reset HEAD~1`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reset HEAD~1
```

`git reset --soft HEAD~1`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reset --soft HEAD~1
```

PARTE 6 REMOTO(10 comandos)

`git push`

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git push --set-upstream origin nuevo-nombre
info: please complete authentication in your browser...
remote: Repository not found.
fatal: repository 'https://github.com/juan/cafe-aurora.git/' not found
```

`git push -u origin main`



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git push -u origin main
info: please complete authentication in your browser...
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 4 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (13/13), 1.07 KiB | 363.00 KiB/s, done.
Total 13 (delta 3), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (3/3), done.
To https://github.com/jonathanjoya642-prog/tallerGIT.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

git push origin feature/login

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git push origin feature/login
error: src refspec feature/login does not match any
error: failed to push some refs to 'https://github.com/jonathanjoya642-prog/tallerGIT.git'
```

git pull

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git pull
Already up to date.
```

git fetch

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git fetch
```

git fetch --all

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git fetch --all
```

git remote add origin URL

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git remote add origin URL
error: remote origin already exists.
```

git remote remove origin

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git remote remove origin
```



git remote show origin

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git remote show origin
* remote origin
  Fetch URL: https://github.com/jonathanjoya642-prog/tallerGIT.git
  Push URL: https://github.com/jonathanjoya642-prog/tallerGIT.git
  HEAD branch: main
  Remote branches:
    main          new (next fetch will store in remotes/origin)
    nuevo-nombre  new (next fetch will store in remotes/origin)
  Local refs configured for 'git push':
    main          pushes to main          (up to date)
    nuevo-nombre  pushes to nuevo-nombre (up to date)
```

git ls-remote

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git ls-remote
From https://github.com/jonathanjoya642-prog/tallerGIT.git
630d23d5e21bfbdf6976748345d72097946b06f      HEAD
630d23d5e21bfbdf6976748345d72097946b06f      refs/heads/main
5c6d46211319dadf53c3e6303b22ba79820bbb9c      refs/heads/nuevo-nombre
```

PARTE 7 STASH(6 comandos)

git stash

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash
Saved working directory and index state WIP on nuevo-nombre: 5c6d462 Agregar archivo.txt por primera vez
```

git stash list

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash list
stash@{0}: WIP on nuevo-nombre: 5c6d462 Agregar archivo.txt por primera vez
stash@{1}: WIP on nuevo-nombre: 630d23d Mi primer commit con archivo.txt
```

git stash pop

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash pop
Already up to date.
On branch nuevo-nombre
nothing to commit, working tree clean
Dropped refs/stash@{0} (1a41da749dd9f5f8899cf5c8bcc471e1eed18532)
```

git stash apply



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash apply
On branch nuevo-nombre
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   nuevo.txt
```

git stash drop

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash drop
Dropped refs/stash@{0} (e3e30d3489181d1448c9614db89635dc9d3d3023)
```

git stash clear

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git stash clear
```

PARTE 8 RECUPERACIÓN Y DEBUG

(10 comandos)

git reflog

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reflog
5c6d462 (HEAD -> nuevo-nombre) HEAD@{0}: reset: moving to HEAD
5c6d462 (HEAD -> nuevo-nombre) HEAD@{1}: reset: moving to HEAD~1
630d23d (main, feature/contacto0) HEAD@{2}: reset: moving to HEAD~1
0176e3b HEAD@{3}: revert: Revert "Mi primer commit con archivo.txt"
630d23d (main, feature/contacto0) HEAD@{4}: rebase (finish): returning to refs/heads/nuevo-nombre
630d23d (main, feature/contacto0) HEAD@{5}: rebase (start): checkout HEAD~3
630d23d (main, feature/contacto0) HEAD@{6}: reset: moving to HEAD
630d23d (main, feature/contacto0) HEAD@{7}: Branch: renamed refs/heads/feature/login to refs/heads/nuevo-nombre
630d23d (main, feature/contacto0) HEAD@{8}: checkout: moving from main to feature/login
630d23d (main, feature/contacto0) HEAD@{9}: checkout: moving from feature/login to main
630d23d (main, feature/contacto0) HEAD@{10}: checkout: moving from main to feature/login
630d23d (main, feature/contacto0) HEAD@{11}: checkout: moving from main to main
630d23d (main, feature/contacto0) HEAD@{12}: checkout: moving from feature/contacto0 to main
630d23d (main, feature/contacto0) HEAD@{13}: checkout: moving from main to feature/contacto0
630d23d (main, feature/contacto0) HEAD@{14}: commit: Mi primer commit con archivo.txt
5c6d462 (HEAD -> nuevo-nombre) HEAD@{15}: commit: Agregar archivo.txt por primera vez
b27123d HEAD@{16}: commit: mensaje
181bbe9 HEAD@{17}: revert: Revert "primer commit"
41b7a35 (tag: v1.1, tag: v1.0) HEAD@{18}: commit (initial): primer commit
```

git reset --hard HEAD

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reset --hard HEAD
HEAD is now at 5c6d462 Agregar archivo.txt por primera vez
```

git reset --mixed HEAD



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reset --mixed HEAD
```

git reset --soft HEAD

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git reset --soft HEAD
```

git fsck

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git fsck
Checking ref database: 100% (1/1), done.
Checking object directories: 100% (256/256), done.
dangling blob d6123a2e7f80a5ae449400388e6d1cfdb2dbe846
dangling commit 1a41da749dd9f5f8899cf5c8bcc471e1eed18532
dangling tree 5af4b175485fed385b2cba198870b374ace2f3e9
dangling commit e3e30d3489181d1448c9614db89635dc9d3d3023
```

git gc

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git gc
Enumerating objects: 15, done.
Counting objects: 100% (15/15), done.
Delta compression using up to 4 threads
Compressing objects: 100% (11/11), done.
Writing objects: 100% (15/15), done.
Total 15 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Enumerating cruft objects: 9, done.
Traversing cruft objects: 14, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 4 threads
Compressing objects: 100% (7/7), done.
Writing objects: 100% (9/9), done.
Total 9 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
```

git blame archivo.txt

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git blame archivo.txt
5c6d4621 (jonathan joya 2026-05-31 14:41:26 -0500 1) g9it
```

git show commit-id



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git show 5c6d462
commit 5c6d46211319dadf53c3e6303b22ba79820bbb9c (HEAD -> nuevo-nombre)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date: Sun May 31 14:41:26 2026 -0500

    Agregar archivo.txt por primera vez

diff --git a/archivo.txt b/archivo.txt
new file mode 100644
index 0000000..d3b3b0b
--- /dev/null
+++ b/archivo.txt
@@ -0,0 +1 @@
+g9it
\ No newline at end of file
```

git bisect start

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git bisect start
status: waiting for both good and bad commits
```

git bisect bad

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre|BISECTING)
$ git bisect bad
status: waiting for good commit(s), bad commit known
```

git bisect Good

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre|BISECTING)
$ git bisect good 41b7a35
Bisecting: 0 revisions left to test after this (roughly 1 step)
[b27123d1965c2d8493d6e98c916e5777c8d1ac49] mensaje
```

PARTE 9 VERSIONES (5 comandos)

git tag v1.0

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git tag v1.0
fatal: tag 'v1.0' already exists
```

git tag -a v1.1 -m "release"

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git tag -a v1.2 -m "mi nueva version"
```

git tag



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git tag
v1.0
v1.1
v1.2
git push -tags
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git push --tags
Enumerating objects: 2, done.
Counting objects: 100% (2/2), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (2/2), 307 bytes | 307.00 KiB/s, done.
Total 2 (delta 0), reused 1 (delta 0), pack-reused 0 (from 0)
To https://github.com/jonathanjoya642-prog/tallerGIT.git
 * [new tag]          v1.0 -> v1.0
 * [new tag]          v1.1 -> v1.1
 * [new tag]          v1.2 -> v1.2
```

git show v1.0

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git show v1.0
commit 41b7a35d1b600830ed2f4a3923fbddcee991ed81 (tag: v1.1, tag: v1.0)
Author: jonathan joya <jonathanjoya642@gmail.com>
Date: Sun May 31 13:32:20 2026 -0500

    primer commit

diff --git a/README.md b/README.md
new file mode 100644
index 0000000..93dee76
--- /dev/null
+++ b/README.md
@@ -0,0 +1,5 @@
+autores:Anyelo, jeann,camilo,jonathan
+
+instagram: @cafeaurora
+
+antacto:1234567890
diff --git a/contacto.txt b/contacto.txt
new file mode 100644
index 0000000..e69de29
diff --git a/menu-cafeteria.txt b/menu-cafeteria.txt
new file mode 100644
index 0000000..408c5a3
--- /dev/null
+++ b/menu-cafeteria.txt
@@ -0,0 +1 @@
+masssss
\ No newline at end of file

jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$
```



PARTE 10 LIMPIEZA (5 comandos)

git clean -n

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git clean -n
```

git clean -f

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git clean -f
```

git clean -fd

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git clean -fd
```

git prune

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git prune
```

git gc -aggressive

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git gc -aggressive
usage: git gc [<options>]

    -q, --[no-]quiet           suppress progress reporting
    --[no-]prune[=<date>]     prune unreferenced objects
    --[no-]cruft               pack unreferenced objects separately
    --max-cruft-size <n>      with --cruft, limit the size of new cruft packs
    --[no-]aggressive         be more thorough (increased runtime)
    --[no-]auto               enable auto-gc mode
    --[no-]detach              perform garbage collection in the background
    --[no-]force               force running gc even if there may be another gc running
    --[no-]keep-largest-pack   repack all other packs except the largest pack
    --[no-]expire-to <dir>    pack prefix to store a pack containing pruned objects
```

PARTE 11 UTILIDADES AVANZADAS (5 comandos)

git shortlog



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git shortlog
jonathan joya (4):
    primer commit
    Revert "primer commit"
    mensaje
    Agregar archivo.txt por primera vez
```

git archive

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git archive
usage: git archive [<options>] <tree-ish> [<path>...]
    or: git archive --list
    or: git archive --remote <repo> [--exec <cmd>] [<options>] <tree-ish> [<path>...]
    or: git archive --remote <repo> [--exec <cmd>] --list

    --[no-]format <fmt>    archive format
    --[no-]prefix <prefix>
                           prepend prefix to each pathname in the archive
    --[no-]add-file <file>
                           add untracked file to archive
    --[no-]add-virtual-file <path:content>
                           add untracked file to archive
    -o, --[no-]output <file>
                           write the archive to this file
    --[no-]worktree-attributes
                           read .gitattributes in working directory
    -v, --[no-]verbose      report archived files on stderr
    --mtime <time>         set modification time of archive entries
    -NUM                   set compression level

    -l, --[no-]list        list supported archive formats

    --[no-]remote <repo>   retrieve the archive from remote repository <repo>
    --[no-]exec <command>  path to the remote git-upload-archive command
```

git describe

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git describe
v1.2
```

git check-ignore

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git check-ignore mi_archivo.log
```

git update-index



```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git update-index
```

PARTE 12 AYUDA Y CONTROL (5 comandos)

git help

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]

These are common Git commands used in various situations:


start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one


work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index


examine the history and state (see also: git help revisions)
  bisect     Use binary search to find the commit that introduced a bug
  diff       Show changes between commits, commit and working tree, etc
  grep       Print lines matching a pattern
  log        Show commit logs
  show       Show various types of objects
  status     Show the working tree status


grow, mark and tweak your common history
  backfill   Download missing objects in a partial clone
  branch     List, create, or delete branches
  commit     Record changes to the repository
  history    EXPERIMENTAL: Rewrite history
  merge      Join two or more development histories together
  rebase     Reapply commits on top of another base tip
  reset      Set 'HEAD' or the index to a known state
  switch     Switch branches
  tag        Create, list, delete or verify tags


collaborate (see also: git help workflows)
  fetch      Download objects and refs from another repository
  pull       Fetch from and integrate with another repository or a local branch
  push       Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$
```



git help merge

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git help merge
```

git help rebase

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git help rebase
```

git help stash

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git help stash
```

git config --global -l

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git config --global -l
user.name=jonathan joya
user.email=jonathanjoya642@gmail.com
```

PARTE 13 FLUJO REAL DE TRABAJO (EQUIPO)

Ejemplo completo de trabajo profesional:

git pull

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git pull
From https://github.com/jonathanjoya642-prog/tallerGIT
 * [new branch]      main      -> origin/main
 * [new branch]      nuevo-nombre -> origin/nuevo-nombre
There is no tracking information for the current branch.
Please specify which branch you want to merge with.
See git-pull(1) for details.
```

```
git pull <remote> <branch>
```

If you wish to set tracking information for this branch you can do so with:

```
git branch --set-upstream-to=origin/<branch> nuevo-nombre
```

git checkout -b feature/payments

```
jonat@F_Joya MINGW64 ~/tallerGIT (nuevo-nombre)
$ git checkout -b feature/payments
Switched to a new branch 'feature/payments'
```

git add .



```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git add .
```

`git commit -m "Add payments system"`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git commit -m "Add payments system"
On branch feature/payments
nothing to commit, working tree clean
```

`git push origin feature/payments`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git push origin feature/payments
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/payments' on GitHub by visiting:
remote:   https://github.com/jonathanjoya642-prog/tallerGIT/pull/new/feature/payments
remote:
To https://github.com/jonathanjoya642-prog/tallerGIT.git
 * [new branch]   feature/payments -> feature/payments
```

`git fetch`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git fetch
```

`git rebase main`

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git rebase main
Successfully rebased and updated refs/heads/feature/payments.
```



```
git merge feature/payments
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git merge feature/payments
Already up to date.
```

```
git push
```

```
jonat@F_Joya MINGW64 ~/tallerGIT (feature/payments)
$ git push
fatal: The current branch feature/payments has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/payments

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.
```

RESUMEN

En empresas se usa realmente esto:

20% de comandos hacen 80% del trabajo:

- git add
- git commit
- git push
- git pull
- git branch
- git checkout / switch
- git merge
- git stash
- git log
- git reset



IMPORTANTE

Muchos comandos:

- se usan solo en casos extremos (bisect, fsck, gc)
- o en debugging o en equipos grandes

CONCLUSIÓN

Git no se aprende por cantidad de comandos, sino por: entender flujo:

Working Directory → Staging → Commit → Remote



Taller Práctico de Git

Proyecto: Café Aurora

“Aprender Git no es memorizar comandos, es entender cómo trabajan los equipos reales de desarrollo.”

Objetivo del Taller

Construiremos paso a paso el proyecto Café Aurora, una página web sencilla para una cafetería ficticia.

Durante el proceso aprenderás cómo se trabaja con Git en ambientes profesionales usando:

- Commits
- Ramas (branches)
- Merge
- Stash
- Rebase
- Tags
- GitHub y remotos
- Restauración de archivos
- Historial y debugging
- Resolución de conflictos
- Flujo de trabajo colaborativo

Al finalizar el taller ¿qué aprenderás realmente?

- Crear repositorios profesionales
- Controlar versiones de proyectos
- Trabajar en equipo usando GitHub
- Recuperar cambios perdidos
- Resolver errores comunes
- Entender el flujo real de desarrollo en empresas

Escenario del Proyecto

Supón que trabajas como desarrollador en la cafetería digital:

Ing. Víctor Rincón



Café Aurora

Tu equipo necesita crear una pequeña web para mostrar:

- menú
- contacto
- redes sociales
- información del negocio

Estructura inicial

```
cafe-aurora/  
├── index.html  
├── menu.txt  
├── contacto.txt  
└── README.md
```

PARTE 1 Crear el repositorio

1. Inicializar Git

```
git init
```

Convierte la carpeta en un repositorio Git.

Resultado esperado:

```
Initialized empty Git repository
```

2. Verificar estado

```
git status
```

Muestra archivos modificados, nuevos o pendientes.

3. Configurar usuario

```
git config --global user.name "Juan"  
git config --global user.email "juan@email.com"
```

Ver configuración:

```
git config --list
```

Ing. Víctor Rincón



Git guardará quién realiza cada cambio.

PARTE 2 Primer Commit

Crear README.md

Ingresa el siguiente contenido:

```
Proyecto cafetería Cafe Aurora
```

4. Agregar archivo al staging

```
git add README.md
```

Prepara archivos para el commit.

5. Verificar cambios preparados

```
git status
```

Verá:

```
Changes to be committed
```

6. Crear primer commit

```
git commit -m "Primer commit"
```

Guarda una versión oficial del proyecto.

7. Ver historial

```
git log
```

8. Historial resumido

```
git log --oneline
```

PARTE 3 Trabajar con archivos

Crear archivo

Ing. Víctor Rincón



```
menu.txt
```

9. Agregar todos los archivos

```
git add .
```

El punto significa: “todo”.

10. Ver diferencias

Modifica `menu.txt` y ejecuta:

```
git diff
```

Muestra cambios aún no preparados.

11. Guardar cambios

```
git commit -m "Agrega menú inicial"
```

PARTE 4 Ramas (Branches)

12. Ver ramas

```
git branch
```

13. Crear rama

```
git branch feature/contacto
```

Las ramas permiten trabajar sin afectar la versión principal.

14. Cambiar de rama

```
git checkout feature/contacto
```

15. Volver a main

```
git switch main
```

16. Crear y cambiar en un solo comando

```
git checkout -b feature/redes-sociales
```

Ing. Víctor Rincón



PARTE 5 Merge

Agregar:

```
instagram: @cafeaurora
```

Guardar cambios:

```
git add .  
git commit -m "Agrega redes sociales"
```

17. Fusionar ramas

Volver a main:

```
git switch main
```

Fusionar:

```
git merge feature/redes-sociales
```

Une el trabajo de dos ramas.

PARTE 6 Eliminar y mover archivos

18. Eliminar archivos

```
git rm contacto.txt
```

19. Renombrar archivos

```
git mv menu.txt menu-cafeteria.txt
```

20. Confirmar cambios

```
git commit -m "Reorganiza archivos"
```

PARTE 7 Restaurar cambios

Modifica README.md

21. Restaurar archivo

Ing. Víctor Rincón



```
git restore README.md
```

Descarta cambios no guardados.

22. Quitar del staging

```
git restore --staged README.md
```

PARTE 8 Trabajar con GitHub

23. Conectar repositorio remoto

```
git remote add origin https://github.com/juan/cafe-aurora.git
```

24. Ver remotos

```
git remote -v
```

25. Subir proyecto

```
git push -u origin main
```

26. Descargar cambios

```
git pull
```

27. Actualizar referencias remotas

```
git fetch
```


28. Clonar proyecto

```
git clone https://github.com/juan/cafe-aurora.git
```

PARTE 9 Stash

Escenario real: debes cambiar rápido de rama sin perder trabajo.

29. Guardar cambios temporalmente

```
git stash
```

Ing. Víctor Rincón



30. Ver stash guardados

```
git stash list
```

31. Recuperar cambios

```
git stash pop
```

PARTE 10 Historial y debugging

32. Ver detalles de commits

```
git show
```

33. Ver historial interno

```
git reflog
```

Uno de los comandos más poderosos de Git.

34. Ver quién modificó cada línea

```
git blame README.md
```

PARTE 11 Versiones y Releases

35. Crear versión

```
git tag v1.0
```

36. Crear tag anotado

```
git tag -a v1.1 -m "Versión estable"
```

37. Subir tags

```
git push --tags
```

PARTE 12 Reset y Revert

Ing. Víctor Rincón



38. Quitar archivo del staging

```
git reset HEAD README.md
```

39. Deshacer commit sin borrar historial

```
git revert HEAD
```

Ideal para trabajo colaborativo.

PARTE 13 Rebase

40. Reorganizar historial

```
git rebase main
```

Mantiene un historial más limpio.

EXTRA Flujo profesional real

```
git pull  
git switch -c feature/menu  
git add .  
git commit -m "Actualiza menú"  
git push origin feature/menu
```

RESUMEN VISUAL DEL FLUJO GIT

Working Directory



git add



Staging Area



git commit



Repositorio local



git push



GitHub

LOS COMANDOS MÁS IMPORTANTES

El 80% del trabajo real usa:

Ing. Víctor Rincón



```
git add
git commit
git push
git pull
git branch
git switch
git merge
git stash
git log
git reset
```

IMPORTANTE

Muchos comandos avanzados como `git bisect`, `git fsck`, `git gc`
Se usan principalmente para:

- debugging
- mantenimiento
- proyectos grandes
- recuperación avanzada

CONCLUSIÓN

Git no se aprende memorizando 100 comandos.

Git se aprende entendiendo el flujo:

```
Working Directory
  ↓
Staging
  ↓
Commit
  ↓
Remote Repository
```

RETO FINAL

Proyecto: E-commerce Café Aurora Pro

Trabajarás como si estuvieras en una empresa real usando:

- GitHub
- ramas por funcionalidades
- corrección de bugs

Ing. Víctor Rincón



- trabajo en equipo
- rebase y merge
- debugging
- recuperación de errores
- versiones estables

Desafío Final

¿Serías capaz de:

- crear nuevas funcionalidades?
- resolver conflictos?
- recuperar commits eliminados?
- trabajar en equipo sin dañar el proyecto?
- desplegar versiones estables?

Meta del Taller

Al terminar este laboratorio podrás decir:

- “Sé usar Git en proyectos reales.”
- “Entiendo cómo trabajan los equipos de desarrollo.”
- “Puedo colaborar profesionalmente usando Git y GitHub.”

Objetivo del proyecto

Construir una tienda online para:

- vender café
- mostrar catálogo
- gestionar pedidos
- manejar usuarios
- integrar pagos
- publicar releases

ARQUITECTURA DEL REPOSITORIO

```
cafe-aurora/  
├── frontend/  
├── backend/  
├── docs/  
└── tests/
```

Ing. Víctor Rincón



```
├── .gitignore  
└── README.md
```

FASE 1 INICIO DEL PROYECTO

1. Clonar o iniciar repo

```
git clone https://github.com/cafe/aurora.git  
cd cafe-aurora
```

o si empiezas desde cero:

```
git init
```

2. Configuración del equipo

```
git config --global user.name "Developer 1"  
git config --global user.email "dev1@cafe.com"
```

3. Conectar remoto

```
git remote add origin https://github.com/cafe/aurora.git
```

FASE 2 PRIMER RELEASE BASE

4. Crear estructura inicial

```
git status
```

5. Agregar todo

```
git add .
```

6. Primer commit

```
git commit -m "chore: estructura inicial del proyecto"
```

7. Subir a GitHub

```
git push -u origin main
```

FASE 3 DESARROLLO POR RAMAS Ing.

Víctor Rincón



8. Crear rama de desarrollo

```
git checkout -b develop
```

9. Crear feature catálogo

```
git checkout -b feature/catalogo
```

10. Trabajar catálogo

Se agregan productos:

- café colombiano
- café espresso
- café premium

11. Guardar cambios

```
git add .  
git commit -m "feat: catálogo de productos inicial"
```

12. Subir rama

```
git push origin feature/catalogo
```

13. Merge a develop

```
git checkout develop  
git merge feature/catalogo
```

FASE 4 SISTEMA DE PAGOS

14. Crear rama

```
git checkout -b feature/pagos
```

15. Cambios en pagos

Se agregan:

- PayPal
- tarjeta
- Stripe

Ing. Víctor Rincón



16. Guardar cambios

```
git add .  
git commit -m "feat: integración de pagos"
```

17. Subir rama

```
git push origin feature/pagos
```

FASE 5 CONFLICTO REAL (IMPORTANTE)

Dos personas modifican el mismo archivo:

```
checkout.html
```

18. Merge conflict

```
git checkout develop  
git merge feature/pagos
```

Conflicto aparece.

19. Ver estado

```
git status
```

20. Resolver conflicto manualmente

Editas archivo y decides versión correcta.

21. Marcar como resuelto

```
git add checkout.html  
git commit -m "fix: resolve merge conflict en pagos"
```

FASE 6 RELEASE

22. Crear rama release

```
git checkout -b release/v1.0
```

Ing. Víctor Rincón



23. Ajustes finales

- bugs menores
- UI
- performance

24. Commit final

```
git commit -am "release: versión 1.0 lista"
```


25. Merge a main

```
git checkout main  
git merge release/v1.0
```

26. Tag de versión

```
git tag -a v1.0 -m "Primera versión estable"
```

27. Subir todo

```
git push origin main  
git push --tags
```

FASE 7 TRABAJO DE EMERGENCIA (HOTFIX)

28. Bug crítico en pagos

```
git checkout -b hotfix/pagos-error
```

29. Fix rápido

```
git commit -am "hotfix: error crítico en checkout"
```

30. Merge inmediato

```
git checkout main  
git merge hotfix/pagos-error
```

FASE 8 HISTORIAL Y CONTROL Ing. Víctor

Rincón



31. Ver historial

```
git log --oneline --graph
```

32. Ver cambios de un commit

```
git show HEAD
```

33. Quién modificó qué

```
git blame backend/payments.js
```

34. Recuperar estado anterior

```
git reflog
```

FASE 9 STASH (trabajo urgente)

35. Guardar cambios temporales

```
git stash
```

36. Volver cambios

```
git stash pop
```

FASE 10 REBASE (historial limpio)

37. Reorganizar commits

```
git rebase main
```

38. Rebase interactivo

```
git rebase -i HEAD~3
```

FASE 11 DEBUG AVANZADO

39. Buscar bug

```
git bisect start  
git bisect bad  
git bisect good v0.9
```



Ing. Víctor Rincón

40. Limpiar repo

```
git gc
git fsck
```

FLUJO FINAL DEL PROYECTO

```
feature → develop → release → main → tag
    ↓ ↓ ↓ ↓
testing staging producción versión
```

LO QUE APRENDISTE REALIZANDO EL ANTERIOR PROYECTO

Flujo real de empresa

- feature branches
- develop branch
- release branch
- hotfix branch

Trabajo en equipo

- merges
- conflictos
- rebase

Producción real

- tags
- versiones
- hotfix

Control total

- logs

Ing. Víctor Rincón



- reflog

- blame
- bisect